Final Project Design Phase

Computer Science - Week 11

Oct 3, 2026 - Version 0.0.4

For complete instructions and project ideas, refer to the assignment in Canvas. Specifically, review the final project requirements before you continue below. If you have not reviewed the project requirements in Canvas (you might have to go to the modules view and check the current or past week's content), you will not be able to properly complete the parts below. In addition, it may be helpful to review the information on this Drafter documentation page.

0) Project Planning and Preparation

Successful programmers spend time planning before they begin coding. *After* reviewing the project requirements, answer the following questions to help guide your design and development process.

A. What concepts from class or previous assignments might help you complete this project? Which programming concepts (loops, conditionals, lists, classes, etc.) do you expect to use? What programming skills do you want to improve while completing this project?

I will use a state dataclass with fields like generation capacity and load that are updated when the user makes decisions like building power plants or data centers. I will use lists of ints to represent generation/energy use by hour for a given day. For intermittent sources like wind, these lists may be randomized using the random module. Route functions to build generation, etc will be called depending on user input and update the state. I want to improve on nested data structures when building this program.

B. Which parts of the project do you expect to be the easiest for you? The hardest? Why?

I expect that the easiest part will be the core functions that will simply check whether gen is sufficient to cover load. The hard part will be layering in dynamics like weather that create variability and making the simulation a realistic model.

C. Even though you have not yet designed the final project, estimate which tasks will likely require the most time and effort? Which features or components should be implemented first?

I would start with creating the initial dataclasses and lists that will be used throughout the program. The tasks that will be most time consuming will be creating realistic scenarios to run the grid against in order to test if it withstands.

D. What distractions might interfere with your work and how will you minimize them? What resources will you need available while working? What questions, if any, do you still have about the assignment requirements?

I will rely on drafter documentation, past assignments, AI, as well as energy-specific resources that will help me make a model that reflects real world conditions.

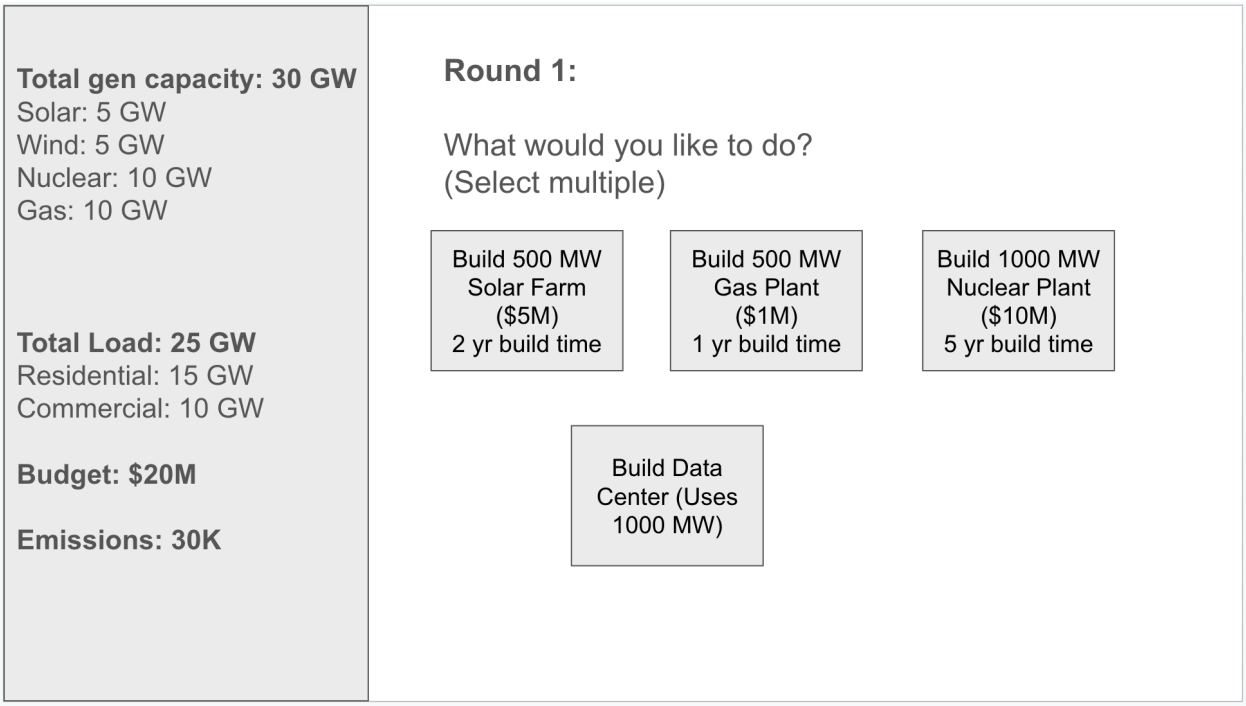
1) Site Diagram

Create a mock-up of your planned website. We suggest you make the mockup on paper, although you can also use software like PowerPoint or Slides to make a mockup. There is other design software that could also be used, but we strongly encourage you to keep it simple. Do not spend more than 15-30 minutes detailing the mock-up. Your goal should be to make a very quick and dirty diagram, to show that you have some kind of idea.

We recommend drawing each page as a rectangle, and laying out the text, buttons, and other components inside. Then, use arrows to indicate links between pages. Label each page the way you will name each function, and then put labels on the arrows to indicate what the button links. Also write down any parameters that should come into the pages.

In addition to the pages and routes, write down what fields your `State` will have, both their names and types. Think clearly about what data will need to be persistent across the entire website.

Add photo(s) or screenshot(s) of your website mock-up and labeled diagram on the next page.



2) Implementing Requirements

This table should serve as a draft for how you will implement the requirements into your final project. You may change your ideas later on, but your final site must fulfill all the requirements.

Requirement	**Implementation**
Purpose/Functionality A 1-2 sentence description of the purpose or functionality of your site	The site will allow the user to make choices to build different types of power plants and test if the grid will withstand

State Dataclass Attributes Minimum of 4 attributes, one of which must be a list/dict of instances of your second dataclass. Briefly describe each attribute, its type, and how it will be meaningfully modified throughout the site.	budget: int - Budget constraint that user can use to build plants plants: list[Plant] - List of plants that is updated when user builds a new plant demand: list[int] - List of 24 hours representing energy demand for each hour in day shortfall_hours: int - Number of hours that grid fails to meet demand when simulation is run emissions: int - Estimated yearly carbon emissions based on user's grid
Second Dataclass Your second dataclass, including its attributes, their types, and how they will be meaningfully modified (if they are modified)	@dataclass class Plant: kind: str capacity_mw: int Build_time: int cost: int availability: list[float] emissions: float kind: type of generation (ie solar, wind, nuclear) capacity_mw: max generation capability of plant Build_time: number of years it takes to build plant (each round will be a year)

	Cost: cost to build plant Availability: list of 24 floats from 0.0-1.0 representing portion of plant's capacity available at each hour (ie solar will only be 1.0 when it is very sunny) Emissions: yearly emissions per mw of capacity
Routes (minimum 7) The names of your route functions	Index Build_menu Buy_plant Build_load Retire_plant Run_simulation Results
Pages (minimum 5) A brief description of the pages in your site	Home page: default screen when opening with instructions on how to play, start button Build menu: Main screen where user can make decisions to build, button to advance to running the simulation Level results page: Page shown at the end of level showing how the grid held up Not enough budget page: Shown if user tries to build something that would go over their budget Game over page: Shown if the user fails (if the grid doesn't withstand or they don't meet emissions target) Victory page: Shown if the user beats all the levels

User Input (minimum 3) Description of user input components (TextBox, TextArea, CheckBox, or SelectBox) and their purposes For more information on these input types, see Drafter Documentation.	Plant select box: user selects which type of generation they want to build Plant quantity text box: player inputs how many MW of that gen type they want to add to the grid Load select box: user selects if they want to add load to the grid Load quantity text box: user inputs how many MW of load they want to build
If Statements (minimum 3) Brief description of what will be evaluated in your if statements and how they will meaningfully change the logic of the site	Run_simulation function: will have an if statement checking whether total generation >= total load, it will return the level results page if true, the game over page if false Build_plant function: will have an if statement checking if the cost of building the plant selected by the user is <= the budget. If true, the function will continue, if false it will return an error page Run_simulation function: will have an if statement checking if the carbon emissions target is met by the user and return the failure page if not (this will probably be in a helper function)
Loop (minimum 1) This loop must iterate over the list/dict of dataclass instances in your State attribute. Description of this loop and why it is meaningful to the site	Run_simulation function: will have a for loop that loops through each hour in the demand list and compare it to the total generation that the user has on their grid

LLM Conversation

If you used an LLM in any part of this work, copy and paste every part of that conversation below. Otherwise, you may be given a zero for the assignment. Obviously, these words will not all be yours, and this is the exception to the requirement at the top

of this assignment that all responses be in your own words. If you did not use an LLM to assist you in this assignment, simply say so below.

Claude

Here's a ranked list of the routes, with the essential ones first.

Essential

index is the landing page with a short explanation of the game and a start button.

build_menu is the hub of the whole program. It shows the remaining budget, the plants built so far, and a button for each plant type.

buy_plant takes a plant type, checks the budget, appends a Plant to the list, subtracts the cost, and sends the player back to build_menu.

choose_event lets the player pick or see the stress event (none, heat wave, plant outage) before the run.

run_simulation loops through the 24 hours, fills in shortfall_hours and total_cost, then goes to results.

results shows the chart, the scorecard, and whether the grid survived.

Strongly recommended

not_enough_budget is a short error page for failed purchases. It keeps buy_plant clean and gives you a second branch to test.

reset_game (or play_again) restores the starting State so the player can try a new build.

Optional if time allows

remove_plant lets the player undo a purchase and get the money back.

how_it_works is a static page explaining the plant types and demand shape.

Will this work?

Yes, the structure is sound. A few things to watch for.

Every route needs to end by returning a Page. In Drafter, a route that only changes State still has to return a page, so buy_plant should hand back the build menu rather than finishing silently.

Keep run_simulation and results separate. If the calculation lives inside results, it reruns every time the page reloads. Doing it in run_simulation and storing the numbers

in State avoids that.

Watch the demand list. If the heat wave multiplies the demand in State and the player replays, the multiplier compounds. Either rebuild the demand list in reset_game or apply the event only on a copy during the run.

Plan your navigation. The flow is index, build_menu, choose_event, run_simulation, results, then back through reset_game to build_menu. Sketching that as a simple flowchart will catch any dead ends before you write code.

For the chart, check the Drafter docs for how it displays matplotlib plots, since that's the one piece I'd verify before committing to it.

Help me brainstorm the pages i will need in my program, i will need at least 5

Pages and routes aren't one to one. Routes like buy_plant, run_simulation, and reset_game do work and then return a page that already exists, so you need fewer pages than routes.

Essential pages

Home page. A short pitch for the game, a quick note on what the player is trying to do (keep the lights on for 24 hours), and a start button.

Build menu page. This is the one players will see most. It shows remaining budget, a button for each plant type with its cost and capacity, and the list of plants built so far.

Event page. The player picks or is shown the stress event before the run. It also works as a natural "are you done building?" checkpoint.

Results page. The supply versus demand chart, the scorecard (shortfall hours, total cost, event that hit), and a clear survived or blacked out message. Include a play again button.

Not enough budget page. A short error page that tells the player what they couldn't afford and sends them back to the build menu.

Strongly recommended

Grid review page. A summary of everything built plus the demand shape, shown right before the run. It gives players one last chance to adjust, and it makes the flow feel less abrupt than going straight from building to results.

How it works page. A static page explaining the plant types, like why solar disappears at night and why nuclear is flat. It can link from the home page and the build menu.

Flow to sketch

Home leads to build menu, which loops with the budget error page, then goes to the event page, the grid review page, and finally results. Results loops back to the build menu through play again.

If you want to hit exactly five, go with the essentials and fold the grid review into the event page. If time is tight, the how it works page is the easiest one to cut or merge into the home page.

im thinking of making this a multi-level game, where each level is a year and player advances i the grid withstands the year. (i will still only test it against 1-2 of the worst days of the year) this way the player will have to choose between plants that take 1 yr vs 3 yrs to build etc. I also want to add an emissions component where the player fails if they dont reach an emissions target. how will this affect the routes and pages i need

Settling on how to phrase the style constraints

Good change. It adds depth without needing any market modeling, and it mostly adds to your structure rather than rewriting it.

What changes in State first

Plant needs a year it comes online and an emissions rate. State needs the current year, the emissions target for that year, and probably a yearly budget income and a demand growth factor so data centers make each year harder. These drive everything below.

Pages, ranked by how much the change affects them

Build menu. This changes the most. It now needs to show the current year, the emissions target, and two lists, plants online and plants under construction with their completion year. Each plant button should show build time next to cost.

Year results page. This replaces your old results page. It shows both checks (grid reliability and emissions) side by side, so the player can see which one they passed or failed. A pass leads to a “next year” button.

Game over page. New and essential. It says which target you missed and in what year, with a play again button. Splitting this from the year results page keeps the pass and fail branches clean.

Victory page. New and cheap. It shows after the final year is cleared, with a short summary of the final grid.

Home, event, and budget error pages. These stay the same. The event page just

needs to pick one of the worst days for that year.

The how it works page is now worth keeping, since build times and emissions need explaining.

Routes, what is new or different

advance_year (new). Increases the year, adds yearly income to the budget, raises demand, updates the emissions target, and returns the build menu. This is the route that makes the game loop.

run_simulation (changed). Now has three outcomes instead of one. It should only count plants whose online year is at or before the current year. Then it checks shortfall and emissions, and returns the year results page, the game over page, or the victory page depending on the result.

buy_plant (changed). Records the completion year instead of making the plant active right away.

reset_game (changed). Has to reset the year too, along with budget, plants, and demand.

You could skip a separate game over route and handle it inside run_simulation, which keeps the route count the same.

Things to watch

Filter by online year inside the simulation loop rather than removing plants from the list. Otherwise you lose track of what is under construction.

Since there is no dispatch, emissions can just be capacity times profile times an emissions rate, summed across online plants. That keeps it simple and still makes gas versus solar a real tradeoff.

Test the first year by hand. If a 3 year plant can never finish before the player fails, the build time choice feels pointless. The early targets should be forgiving enough that a long build can still pay off.

help me brainstorm user input in this program, i will need 3 instances of this.

Listing the Drafter components for input fields

Here are three that fit your game, ranked by how much they add.

Quantity TextBox on the build menu. Instead of one button per single plant, the player types how many of a plant type to build. This is your best one because it forces real input handling. The value arrives as a string, so buy_plant has to convert it to an

integer, reject blanks, letters, zero, and negatives, and then check the total cost against the budget. Bad input and over-budget input can both send the player to your budget error page, or you can add a short “invalid input” message there.

SelectBox for the stress event. On the event page, the player picks which worst day to test against, like heat wave, plant outage, or a calm day. A dropdown fits because the options are fixed, so you skip validation and the choice goes straight into the event attribute in State. If you want it to feel like a decision, tie it to risk and reward, such as a higher budget bonus for choosing the harder day.

CheckBox or TextBox on the home page. You have two good options here.

A player name TextBox that shows up on the victory and game over pages. It’s trivial to build and makes the end screens feel personal. It needs a default if the field is left empty.

A “hard mode” CheckBox that tightens the emissions targets or raises demand growth. It’s a boolean in State that your advance_year route reads, so it’s a nice way to show input affecting game logic and not only display.

If you want my pick, go with the quantity box, the event dropdown, and the hard mode checkbox. They each affect the simulation in a different way, and they give you three different input types to show off.

One thing to decide early is where the plant type comes from when the player uses the quantity box. The simplest approach is a SelectBox for plant type next to the quantity TextBox, with a single buy button that passes both values to buy_plant. That would give you a fourth input for free.

i will need 3 if statements in my program, help me brainstorm where i will need these

Checking my prose for colon-free style

Here are my picks, ranked by how much they matter to the game.

Purchase validation in buy_plant. This one needs the most branching. If the quantity isn’t a valid positive whole number, send the player to the error page. Otherwise, if the total cost is more than the remaining budget, also send them to the error page. If neither problem applies, add the plants and subtract the cost. An if/elif/else chain fits here.

Outcome check in run_simulation. After the day is simulated, if shortfall hours are above zero or emissions are over the target, return the game over page. Otherwise, if the current year is the final year, return the victory page. Otherwise, return the year results page. This is the decision point of your whole game, so it’s the most important if statement to get right.

Online year filter inside the simulation loop. For each plant, if its online year is at or

before the current year, add its output and emissions to that hour. Plants still under construction get skipped. It's small, but it's what makes build times matter.

Good backups if you want more

In advance_year, if hard mode is checked, use the tougher demand growth and emissions target.

In the event logic, if the event is a heat wave, multiply afternoon demand by 1.2, and if it's a plant outage, zero out one plant.

On the end pages, if the player name box was left empty, fall back to a default name.

If you want to show off, the first one is the best candidate because it can use if, elif, and else together in a way that's easy to explain in your write-up.

i will need one loop in my program that iterates over the list/dict of dataclass instances in my State attribute. help me brainstorm how to incorporate this

Checking the formatting preferences for the answer

Here are my picks, ranked by how much the loop does for the game.

Simulation loop in run_simulation. This is the best fit because the loop does real work. Go through each Plant in the list, skip it if it isn't online yet, and add its hourly output into a 24 item supply list and its emissions into a running total. Putting the plant loop on the outside and the 24 hour pass on the inside means you only loop over the State list once and build the supply curve as you go. Your online year if statement lives inside this same loop, so it counts toward both requirements.

Build menu display. Loop over the plants and create one line of text per plant showing its type, capacity, and online year. You can sort it into two groups by checking the online year, one for plants running now and one for plants under construction. It's simpler than the simulation loop, but the player sees the result on screen every turn.

Scorecard totals. Loop over the plants to add up total capacity, total build cost, and capacity per plant type for the year results page. It's a nice way to show players what their grid looks like, and it's easy to test since you can check the numbers by hand.

Under construction countdown. In advance_year, loop over the plants to find any that finish this year and show a "new plant online" message on the build menu. It's a small touch that makes the year change feel alive.

If you only want one loop, go with number 1. It's the heart of the program, and it gives you a clear way to explain in your write-up why a loop is needed. Since the player can build any number of plants, you can't hardcode the math, and the loop handles whatever size the list happens to be.

Claude is AI and can make mistakes. Please double-check responses.

Claude finished the response

Final Submission

When you are finished, download this file as a Word Document (docx) and upload the file to the appropriate assignment on Canvas.